Assignment Report

SFWRENG 2AA4 (2026W)

Assignment 2

Author(s)

Shifa Zaman	zamans27@mcmaster.ca	zamans27
Syaan Merchant	merchs8@mcmaster.ca	merchs8
Sanika Surose	suroses@mcmaster.ca	suroses
Lakshmi Saranya Alamanda	alamandl@mcmaster.ca	alamandl

GitHub URL

`https://github.com/syaanmerchant/Settler-of-Catan_2aa4.git`

March 19, 2026

Original template credits: Samuel Zurowski and Charles Barone. Adapted for McMaster SFWRENG by Dr. Istvan David.

1 Executive Summary

This project extended our Settlers of Catan simulator to support human gameplay, trading, robber mechanics, step-based execution, and external visualization, while preserving strong software design principles and system stability.

To prepare for change (Task 1), we implemented 15 focused JUnit unit tests covering core mechanics such as resource management, build validation, settlement rules, and resource production. These tests were organized into structured suites and executed using Maven, allowing us to quickly detect regressions as the system evolved.

In Task 2, we evolved the original domain model to incorporate new requirements without disrupting the core simulation logic. Instead of modifying existing classes extensively, we introduced new components with clear responsibilities, including a HumanPlayer subclass, modular approach localized changes and improved maintainability.

We also modelled player actions within a turn using a deterministic finite automaton (DFA), formally defining valid action sequences and preventing illegal transitions. The DFA directly informed the programs control flow and validation logic, ensuring consistency between design and implementation.

Overall, the project demonstrates a structured and design-driven approach to software evolution, resulting in a modular, testable, and extensible simulator that successfully integrates human interaction and visualization support.

2 Requirements Traceability

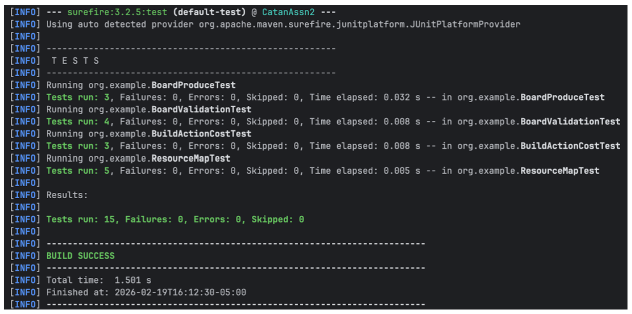
Req ID	Status	Implemented in	Design considerations
R2.1	Implemented	`CommandParser.java`, `HumanPlayer.java`	Human input is read from the command line during the human player's turn. Parsing is handled using regular expressions in `CommandParser`, separating input validation from game logic to maintain single responsibility and improve extensibility.
R2.2	Implemented	`GameStateWriter.java`	Integration with the instructor-provided visualizer is achieved by exporting the current game state in the required JSON format. The design isolates visualization concerns from core game logic to preserve modularity.
R2.3	Implemented	`GameStateWriter.java`	The complete game state is serialized into an external JSON file after relevant state changes. This ensures synchronization with the visualizer while keeping persistence responsibilities separate from simulation logic.
R2.4	Implemented	`Simulator.java`	A step-forward mechanism pauses execution after each turn and waits for a "go" command before proceeding. This functionality is implemented within the simulator's control flow to maintain clear turn sequencing.
R2.5	Implemented	`Board.java`, `Simulator.java`	The Robber mechanism is triggered upon rolling a 7. Card discarding, random robber placement, and random resource stealing are implemented following the simplified rules. The logic is centralized to avoid scattering rule enforcement across multiple classes.
R2.6	Implemented	`Demonstrator.java`	A dedicated demonstrator class with a single `main` method executes one or more sample simulations. The class is documented to clearly illustrate key features such as human interaction, robber activation, step execution, and visualization integration.

3 Testing

Task 1 Reflection

Question 1: How many unit tests did you end up writing? We ended up writing a total of 15 unit tests. The tests are divided into several focused test classes so that each test suite is focused on a key part of the simulator and remains simple to maintain. These tests focus on resource management and payment tracking in ResourceMap, build cost rules in BuildAction, build rules in Board such as road placement and settlement distance, and resource production. This is useful for evolvability because as we will need to modify and extend the simulator for Assignment 2, we can easily re-run the test suite to quickly identify regressions in the most important game invariants and mechanics.

Question 2: If you measured coverage: which tool did you use and what was your coverage? We did not attempt to measure test coverage for this task as we selected the other option available in Task 1, to report with a screenshot of test results instead. We focused on writing meaningful tests that directly test important rules and behaviors and used Maven as the standard, repeatable process to run and prove the tests. The command "mvn test" will run the entire JUnit test suite on any given machine/environment and will provide clear results of the test outcomes. From the screenshot attached below, it can be seen that all 15 tests were successfully run with 0 failures/errors and that the BUILD SUCCESS message was received.



```
[INFO] --- surefire:3.2.5:test (default-test) @ CatanAssn2 ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running org.example.BoardProduceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.032 s -- in org.example.BoardProduceTest
[INFO] Running org.example.BoardValidationTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.008 s -- in org.example.BoardValidationTest
[INFO] Running org.example.BuildActionCostTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.008 s -- in org.example.BuildActionCostTest
[INFO] Running org.example.ResourceMapTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in org.example.ResourceMapTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.501 s
[INFO] Finished at: 2026-02-19T16:12:30-05:00
[INFO] ------------------------------------------------------------------------
```

Figure 1: Success Testing

4 Changing and Extending Design

Task 2 Reflection

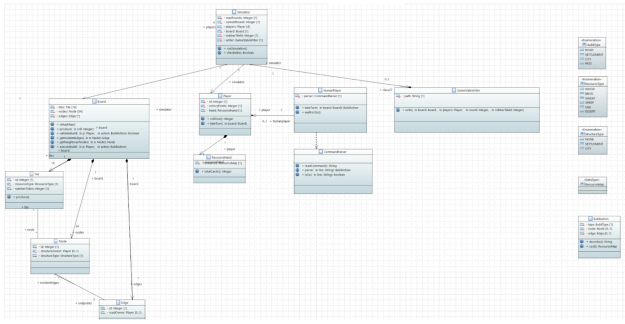


Figure 2: Extended UML Diagam

Question 1: What were the main structural changes in your original design? Was it easy to localize the change and assess its impact compared to directly changing the code (without messing around with the design)? One of the main structural changes we made was in how we expanded the design from Assignment 1 to accommodate human interactions, step-by-step execution, robber handling, and external game state output without having all of these extra features forced into the existing classes of the original simulator. For example, we created a HumanPlayer class that was a subclass of Player in order to accommodate the human participant separately from the other automated participants within the same turn-based framework. We also created a CommandParser class that handled parsing the input from the console and a GameStateWriter class that handled exporting the game state into JSON format for use by the visualizer. Additionally, we made some significant but minor changes to the structure of the program to accommodate the new requirements of Assignment 2 such as storing the robber-related state and expanding the possible actions of a player to include trade-based actions. This was incredibly useful in containing the effects of the new requirements of Assignment 2 because we were able to leave the main elements/logic of the game board, player, and simulation largely untouched and simply add new features through a handful of new additions. In contrast to the direct modification of the code, the update of the design model was useful because it allowed us to think about the placement of responsibilities and the interaction of classes before the actual implementation of the code. This helped to clarify the scope of each change, reduced the risk of the new features being scattered through the code unnecessarily, and provided a clear plan to implement the new requirements into the code.

Question 2: Why was the automaton-based modelling useful and what benefits does it provide compared to the specification in the attached PDF? How did you translate the automaton-based model to program code?

Automation-based modelling was useful as it clearly defines which actions are allowed at each stage of a turn and prevents illegal sequences of actions. The DFA formally defines which actions are allowed in each state and what the next state must be. This removes ambiguity and makes illegal action sequences structurally impossible within the model.

The states: StartTurn, AfterRoll, Building, and EndTurn, represent phases of a player's turn, and the alphabet represents all possible actions. To satisfy the formal definition of a DFA, and ensure no invalid moves are accepted, a trap state called Invalid was introduced so that every state has a defined transition for every action.

The automaton was translated into program code through structured control flow and validation checks. The Simulator manages turn progression, while methods such as takeTurn() and validateBuild() enforce the transition rules. In practice, illegal transitions correspond to rejected commands, mirroring transitions to the trap state in the DFA model. Therefore the implementation directly reflects the structure of the automaton.

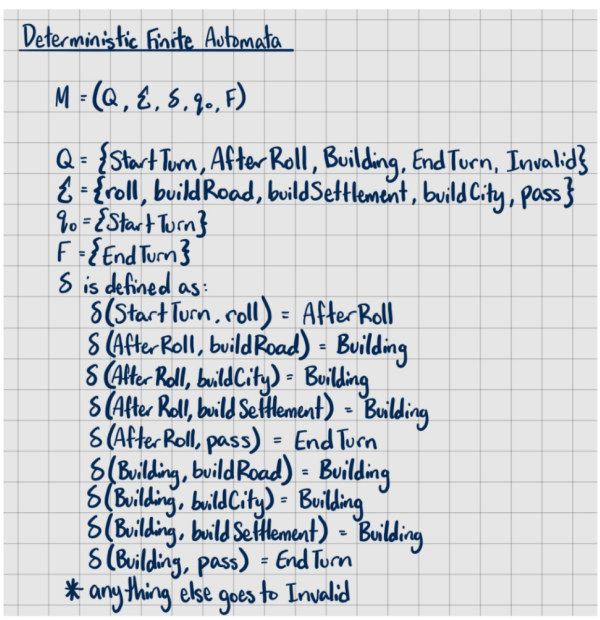


Figure 3: Automata Transitions

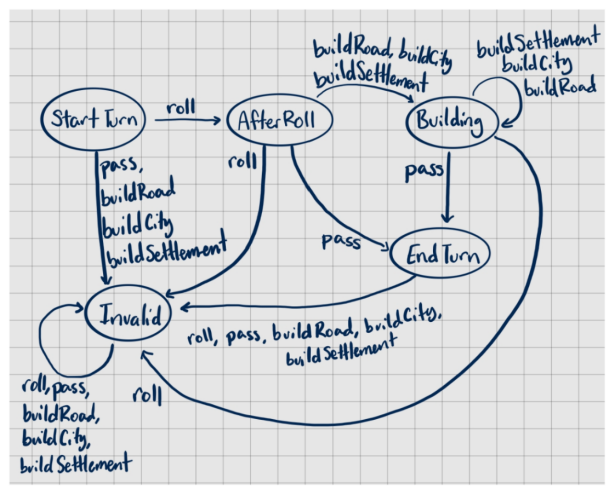


Figure 4: Automation Diagram

5 Testing and QA

Task 3 Reflection

Question 1: Which parts of your system did break and how did you resolve those issues?

The first major break was when we set out to create an abstract base class for Player in order to enable both the MachinePlayer and HumanPlayer. All of our tests and setup code were using new Player(0), so when we made the change, nothing in the project would compile, and all of our JUnit fixtures failed. We fixed this problem by creating MachinePlayer which encapsulated all of the old AI code, and then worked systematically through all call sites and tests (BoardProduceTest, BoardValidationTest, etc.) to replace new Player(...) with MachinePlayer(...). Another large point of stress was integrating the Python visualizer. The visualizer uses a different implementation of the Catan engine (catanatron), and when we first exported our full board state through its board.build_settlement(...) call, we received an Invalid Settlement Placement error for not being connected and not an initial placement, even though our rules believed that our state was, in fact, legal. The subtle difference between the topology that was modelling in Java and the expectations of the catanatron engine caused many crashes and repeated failures until we simplified the GameStateWriter to only provide a stable version of the state.json (empty roads and buildings), while using the base_map.json to provide the layout, thereby meeting both the requirement for integration and maintaining the invariants of the external engine. In addition to this, there were also environment-related delays: we had some problems using pygame with Python 3.14 because we didn't have the SDL headers necessary to compile pygame from the source, and we had to switch from Python 3.14 to Python 3.11 with Homebrew, so we could install the Visualizer using wheels. These problems were not due to a coding error, but they did take time to troubleshoot in order to see the board.

Questions 2: What was the benefit of having unit tests at this stage?

The original unit tests served as guard rails to the refactoring of the core classes and allowed me to confirm that production rules, resource costs, and victory point accounting had not been broken when we changed Player to an abstract class and created MachinePlayer. The test suites for BoardProduceTest, BoardValidationTest, and BuildActionCostTest were used to create those detours. Then we produced the CommandParser, which relied on regular expressions, and created a separate CommandParserTest test suite. This test suite established the contract for what constitutes a valid command (e.g., road between vertices 11 and 12, settlement on vertex 5), as well as documenting edge cases that needed to be handled (e.g., no input, null input, malformed commands, roads with the same endpoints) and providing meaningful error messages when those edge cases were encountered. We would not have known to rerun the entire game (which started being played by other users) to know that command road 5-5 was an acceptable command or that the pass command was not being parsed as a special case without the tests. Additionally, we created the GameStateWriterTest test suite to ensure that any JSON we created had the correct structure and contained the fragment values we expected before we integrated with the Python code; this helped narrow down semantic mismatches for when the visualizer crashed. With unit testing, we had im-

proved self-confidence that our continued evolving of the player and parser designs would not lack any of the core game invariants that we developed in Assignment 1.

Question 3: What were some of the challenges in these iterations (technical or conceptual)? What tools or integrated tool chains would help you in those steps? (You can specify a few abstract ideas, you don't have to name specific tools or techniques.)

The most technically difficult challenges we had with the assignment were due to cross-language integration issues and environment inconsistencies. On the Java side, we conceptually understood how to refactor our Player hierarchy and add the HumanPlayer, but while integrating these changes into the Simulator and Demonstrator, we had to be very careful to maintain the same AI behaviours that we had originally implemented. Getting the visualiser up and running on the Python side turned out to be a more complex task than expected; we had to create a virtual environment for Python 3.11, ensure pygame was installed using pip (with issues compiling SDL2 on macOS), and then install catanatron into an editable environment (which was extensive due to its dependencies) before we was able to get the visualiser working with all of the other Code dependencies. After getting everything installed, we still had to troubleshoot on semantic integration and determine why the visualiser received our board.build_settlement as an illegal state. This exercise forced me to conceptualise how there are two models of "the same" game done independently that can have differences in the definition of invariants. Integrated tooling would have been helpful for me as a Second-Year Student; for instance, a Dockerfile (or Nix flake) that pinpoints. Python/SDL/pygame/catanatron supports a generalized interface that hides OS-based construction from special builds (ex: compiled via py2exe versus pyInstaller) or a basic contract testing harness (takes visualizer and compares output against various known-good state.json file). On the Java side, the way that Cursor provided IDE support and maven made building and running faster was great; maven caught compile errors right away, and JUnit provided almost immediate feedback. But it would have been easier to do a refactor-test cycle if there were some sort of continuous testing upon saving.

Question 4: Elaborate on the OO mechanisms and SOLID principles in your implementation. Are these any different from the ones used in the design? If they are: why?

I utilized OO approaches like inheritance, composition, or encapsulation where it made sense to apply SOLID principles (i.e., based on their usefulness, not just because they exist). The most notable OO modification when contrasting the initial design was the player hierarchy with an abstract Player base class and two derived classes. One of the key ways that the new design adhered to the Liskov Substitution Principle (i.e., any instance of Player would fit in multiple places within the Simulator) is that now we can easily add additional machine players with different strategies. Each class's description is clear under the Single Responsibility Principle: HumanPlayer—only handles user input and validating user input with regard to the game board; MachinePlayer—only makes AI decisions; CommandParser—only parses commands based on regular expressions; and GameStateWriter—only persists JSON data to the output for a visualizer. The Simulator manages all of the rounds and coordinates with all of these classes, rather than Mixing I/O, parsing, and game rules together. Each

class's separation is so much better now than it was in the original version, where the Player class did both state management and AI; and the demonstration logic was jammed into the Demonstrator class. However, there are situations where design implementation diverges the design ideally; e.g., GameStateWriter is expected to export the full game state (with roads and buildings) in a schema that is defined by the visualizer. Because of the way catanatron expected to receive data (heavily mismatched from what the visualizer needed) we hardcoded roads and buildings to simplify things. That is an i.a.t. tradeoff: we still meet the integration requirements and the SRP by allowing the visualizer to use our state.json while running in watch mode, but we did loosen the original goal of visualized player structures. In similar fashion, we have grouped all rules within the Board as well as executed them within BB with respect to SRP. But, the implementation does not allow for easy extension since adding a new build type would require changing switch statements to accommodate. Overall, the design and implementation are both in the same relative direction concerning OO and SOLID but the implementation reflects more pragmatically than the design due to third-party mismatches, time constraints, and keeping the system working reliably even if it means simplifying some responsibilities.

6 Roles and responsibilities

The team members contributed equally to the deliverable.

- TODO: **Shifa Zaman and Lakshmi Saranya Alamanda** contributed to the conceptual design and the implementation of RQ1.1.
- TODO: **Shifa Zaman and Sanika Surose** contributed to the conceptual design and the implementation of RQ1.2.
- TODO: **Shifa Zaman and Lakshmi Saranya Alamanda** contributed to the conceptual design and the implementation of RQ1.3.
- TODO: **Syaan Merchant and Sanika Surose** contributed to the conceptual design and the implementation of RQ1.4.
- TODO: **Syaan Merchant and Sanika Surose** contributed to the conceptual design and the implementation of RQ1.5.
- TODO: **Syaan Merchant and Lakshmi Saranya Alamanda** contributed to the conceptual design and the implementation of RQ1.6.